# Leveraging Large Language Models and Machine Learning for Smart Contract Vulnerability Detection

S M Mostaq Hossain, Amani Altarawneh and Jesse Roberts
*Department of Computer Science*, *Tennessee Technological University*
Cookeville, Tennessee, USA
Email: {shossain42, aaltarawneh, jtroberts}@tntech.edu

2025 IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC) | 979-8-3315-0769-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/CCWC62904.2025.10903869

***Abstract*—As blockchain technology and smart contracts become widely adopted, securing them throughout every stage of the transaction process is essential. The concern of improved security for smart contracts is to find and detect vulnerabilities using classical Machine Learning (ML) models and fine-tuned Large Language Models (LLM). The robustness of such work rests on a labeled smart contract dataset that includes annotated vulnerabilities on which several LLMs alongside various traditional machine learning algorithms such as DistilBERT model is trained and tested. We train and test machine learning algorithms to classify smart contract codes according to vulnerability types in order to compare model performance. Having fine-tuned the LLMs specifically for smart contract code classification should help in getting better results when detecting several types of well-known vulnerabilities, such as Reentrancy, Integer Overflow, Timestamp Dependency and Dangerous Delegatecall. From our initial experimental results, it can be seen that our fine-tuned LLM surpasses the accuracy of any other model by achieving an accuracy of over 90%, and this advances the existing vulnerability detection benchmarks. Such performance provides a great deal of evidence for LLMs' ability to describe the subtle patterns in the code that traditional ML models could miss. Thus, we compared each of the ML and LLM models to give a good overview of each model's strengths, from which we can choose the most effective one for real-world applications in smart contract security. Our research combines machine learning and large language models to provide a rich and interpretable framework for detecting different smart contract vulnerabilities, which lays a foundation for a more secure blockchain ecosystem.**

***Index Terms*—Smart Contract, Large Language Model, Machine Learning, Vulnerability Detection, Fine-tuning, Ethereum.**

## I. Introduction

The increasingly rapid evolution of blockchain technology has fostered the large-scale adoption of smart contracts, defined as self-executing contracts with the terms of the accord explicitly written into code. With various application areas of smart contracts such as finance, supply chain management, and decentralized applications (DApps) rising quite spectacularly, their securing has become a very critical aspect [1]. Security vulnerabilities of smart contracts can lead to dire consequences, including financial losses, breaches of privacy, and systematic risks posed to decentralized systems [2]. The growing prevalence of incidents within the blockchain arena shows the critical consequences of these vulnerabilities, making it clear that effective means of detection for such vulnerabilities need to exist.

Detection of vulnerabilities within smart contracts has mostly relied on static analysis techniques [3] that analyze the code of the contract without executing it. These techniques can detect known vulnerabilities but have faced difficulties in handling rising complications alongside emerging vectors of attack. Thus, alternative approaches, such as machine learning and deep learning, are being explored by researchers to assist in the detection of various types of attacks [4]). These machine learning and deep learning-based methods use training algorithms that can learn from annotated datasets to find patterns that could reveal security holes. This makes it easier to find both known and unknown security problems [5, 6].

The integration of large language models in the context of vulnerability detection for smart contracts signifies a notable advancement in the field of blockchain cybersecurity [7]. Models that command the ability to generate and comprehend the code-in a contextual manner-seem qualified for outperforming the traditional ML models by capturing complicated relationships and semantic information within the contract code. Recent research [8] into the use of fine-tuned Transformer encoder models for effective vulnerability detection finds focus being directed ever more closely toward imposing language models on the enhancement of smart contract security. Building on these advancements, the current work explores how ML and LLM can enhance smart contract vulnerability detection. The main goal of this research is as follows:

(a) Comparative Evaluation: A systematic comparison is carried out, contrasting classical ML models such as LSTM [9] and fine-tuned large language models [8], like DistilBERT [10] and BERT [11], for prioritizing smart contract vulnerability detection and providing paramount effectiveness of LLMs in terms of capturing complex, context-rich patterns.
(b) Detection of Subtle Code Patterns: This study sets forth the capability of LLMs in determining intricate code patterns and dependencies-this classical model misses in vast applications, more prominently with regards to vulnerabilities like reentrancy [12] and integer overflow [13] to show improved pattern recognition.
(c) Understanding the Impact of Fine-Tuning: We evaluate how fine-tuning LLMs on domain-specific smart contract data improves model performance and generalization, giving better precision and recall with respect to complex

979-8-3315-0769-5/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 12:59:33 UTC from IEEE Xplore. Restrictions apply.

vulnerabilities.

(d) Benchmarking and Future Directions: In this work, we establish thorough benchmarks in vulnerability detection, expanding the scope of future research with an eye toward additional dataset expansion, wider LLM explorations, and the use of semi-supervised learning methods for improvement in generalization.

The remaining paper is set out organized as follows: The next section explains the Key Research Questions followed by the Background that depicts the relevant work. The Methodology explains how our approach was executed, including data preparation and model evaluation. Results provide a comparison of classical ML models in the form of LLMs, followed by a Discussion of these findings. Future Work is suggested, and then we conclude with key takeaways in the Conclusions.

## II. Key Research Questions

The Ethereum blockchain ecosystem [14] relies on smart contracts security, the purpose of this research is to fix critical vulnerabilities in these contracts. To clarify and provide a framework for evaluation, following research questions lead the investigation. The questions test how well ML models and LLMs compare, whether an LLM fine-tuned to domain-specific data would benefit and whether LLMs could leverage minute code patterns that standard models wouldn't recognize.

- **RQ-1:** How do classical machine learning models compare to fine-tuned large language models (LLMs) in smart contract vulnerability detection in terms of effectiveness?
- **RQ-2:** Are subtle code patterns and dependencies in smart contracts captured by LLMs, given that those very patterns and dependencies evade classical machine learning models?
- **RQ-3:** What are the effects of the fine-tuned large language model training on labeled smart contract data in enhancing vulnerability detection accuracy, and how do they affect the model's generalization?

## III. Background

Research on smart contract security has been exponentially growing over time. This concern is justified given the potential safety and security risks associated with blockchain technology. The earliest works within this domain focused on static analysis tools [15] for identifying known vulnerabilities without running the associated code. Researchers were able to leverage the static nature of smart contracts to analytically check for breaches and subsequently formalize them using mathematical proof systems. It is currently the approach followed in most vulnerability detection practices. This however requires manual routine inspection, like developers acquiring an understanding of contract logic in order to conduct the audit. Since the approach is "static", there is the natural tendency to obtain more false positives [16] as opposed to dynamic results which is expected to be more precise and hence are assumed to be "dynamic". How do we trust, accept or understand security outcome that we are aware of it generates more false alarms?. This may have a resultant effect in the user acceptance of the security measure.

In order to address these shortcomings enumerated in the static approach to smart contract security checks. Numerous authors have since attempted to build ML models such as decision trees and support vector machines [17] to automatically detect vulnerabilities. These new algorithms take source code as input, extract features from it, build models using the extracted features and then use the model to ultimately classify whether or not a contract has vulnerabilities. Recent research [4] have demonstrated the efficacy of several models in identifying vulnerabilities such as reentrancy and gas limit concerns, attaining notable accuracy rates [12]. Nonetheless, numerous techniques continue to depend significantly on manually built features, constraining their scalability and responsiveness to emerging risks.

Recent research [18], particularly employing Deep Learning for vulnerability detection. Researchers have also found that CNN and LSTM models can extract higher-level representations of smart contracts, leading to improved accuracy detection rates [19]. Deep learning improves code analysis efficiency by automatically learning features from raw inputs, overcoming limitations in classical ML capabilities, particularly in feature engineering. Smart contracts are vulnerable to different attacks that compromise security and reliability [20]. Common issues include Reentrancy (RE) [12], Integer Overflow (IO) [13], Timestamp Dependence (TD) [21], and Dangerous Delegatecall (DD) [22], which can lead to arbitrary code execution. Understanding these weaknesses is crucial for developing efficient detection algorithms.

Encoder-only models like BERT excel at language understanding because they have unfettered access to the full input context, making them ideal candidates for context-aware vulnerability detection in smart contract codes [23]. Researchers who adapt these algorithms to smart contract datasets have observed considerable improvements in vulnerability detection. In the paper [24] Ma et al. introduce an LLM prompt-based framework, iAudit, which aims to fine-tune LLMs for 263 real smart contract auditing through a two-stage decision-making and justification process. The model adds an iterative approach using LLM-based agents to refine the vulnerability explanations. An F1 score of 91.21% is demonstrated together with superiority in detection and consistency relative to existing fine-tuned models such as CodeBERT [25]. Another paper [7] by Boi et al. utilizes the Llama-2-7b model for vulnerability detection and got an accuracy of 59.5% with their Smartbugs Dataset (143 smart contracts). However, their approach demonstrates the potential of using LLMs for vulnerability detection and highlights an opportunity to further improve performance by expanding the dataset and employing enhanced fine-tuning strategies.

## IV. Methodology

This section describes the procedure in an organized way that was adapted to combat the problem of smart contract vulnerability detection using ML and LLMs. It incorporates



Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 12:59:33 UTC from IEEE Xplore. Restrictions apply.

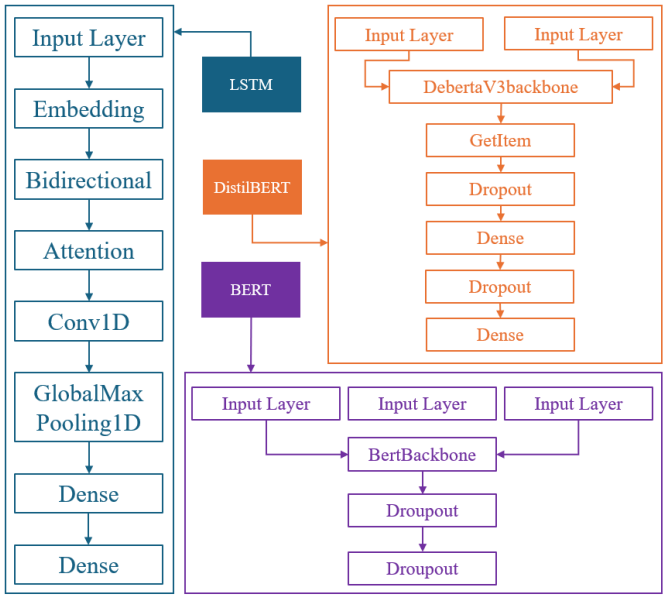



Fig. 1. Customized layer structure of data flow diagram for LSTM, DistilBERT and BERT Models in our experiment.

the dataset employed, models applied and an experimental setup for carrying out the analysis.

### A. Dataset Overview

The primary dataset for this work consisted of an annotated smart contract dataset [26] from Liu et al. This dataset contains smart contracts annotated with various vulnerabilities, providing in-depth analysis and testing. It consists of 2,217 smart contracts with a structured representation of 5 columns: filename, code snippet, vulnerability label, and an encoded label. The dataset includes four major types of smart contract vulnerabilities: Reentrancy, which has the largest collection with 1,218 samples; Integer Overflow with 590 samples; Timestamp Dependency with 312 samples; and Dangerous Delegate Call with 97 samples. Each contract features the Solidity [27] source code along with tagged annotation indicating the type of the specific vulnerability it presents, hence qualifying it for training and evaluation of machine learning and LLM models during vulnerability detection tasks.

### B. Models

The first experiment used an LSTM-multi-layer RNN [28], which is good at processing data sequences. BERT was trained on a large corpus of textual data, including structured programs where DistilBERT is a smaller model distilled from BERT [11]. Figure 1 shows the layered dataflow structure of the models which has been used during our experiment.

- LSTM: The Long short-term memory (LSTMs) analyze linguistic input via recursion, adding each tokenized input to a latent representation that is ultimately used to represent information about the total context. RNN-based techniques are typically outperformed by encoder-only transformer-based techniques in many natural language understanding tasks due to the latter's ability to selectively attend to the entire context without the need for recursion. This makes them more able to capture long-range dependencies [11]. Therefore, we additionally test both BERT and the more efficient DistilBERT in the same application. However, for the LSTM model layer design Bidirectional LSTM with attention and convolution layers have been used [29].
- DistilBERT: A distilled variant of the BERT [10] language model is small, fast, cheap, and, hence, efficient, keeping over 90% of BERT's language understanding efficacy. Efficiency earns it a great deal of applicability in the world of practical issues where computational resources are at a premium.
- BERT: In the experiment, Bidirectional Encoder Representations from Transformers (BERT) [11] model has been used which changed the landscape of natural language processing. The architecture of BERT allows it to learn the deep contextual relationships in the data, making it a natural match for detecting well-known vulnerabilities from the smart contract code.

In this regard, the effectiveness of all models was thoughtfully assessed, followed by experience-based discussions aimed at charting the course to determine the approach to take care of the better smart contract vulnerability detection.

### C. Experimental Setup

Experiments were run on a Google Colab notebook, which is an online Collaborative programming environment best suited for performing Computation heavy tasks. Using Google Colab has the additional benefit of access to GPUs, which speeds up training time for ML and LLM models. The steps of the experimental setup were:

- Data Preparation: It was used to pre-process the data, clean it from code fragments and turn them into a way they can be fed into the models. These comprised of tokenization, normalizing the coding syntax and encoding vulnerabilities to a structured format.
- Hardware and Computational Resources: The experiments utilized Google Colab Pro with an NVIDIA A100 GPU (40 GB RAM), 83.5 GB system RAM, and 58.69 compute units at 8.47 units/hour, ensuring efficient model training and testing for vulnerability detection.
- Model Implementation: The implementation consisted of coding the models in python with appropriate libraries such as TensorFlow and Keras NLP models [30] to go through standard pre-training. Next, the models were trained on this dataset with respective hyperparameter tuning for optimal performance.
- Evaluation metrics: For this work, evaluation metrics such as accuracy, precision, recall, and the F1 score, allowed a quantitative assessment of the performance of the models in performing vulnerability detection. For comparison purposes, the results were documented.
- Iterative testing: It is provided by iterative assessments. As the testing progressed, feedback from the model



Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 12:59:33 UTC from IEEE Xplore. Restrictions apply.

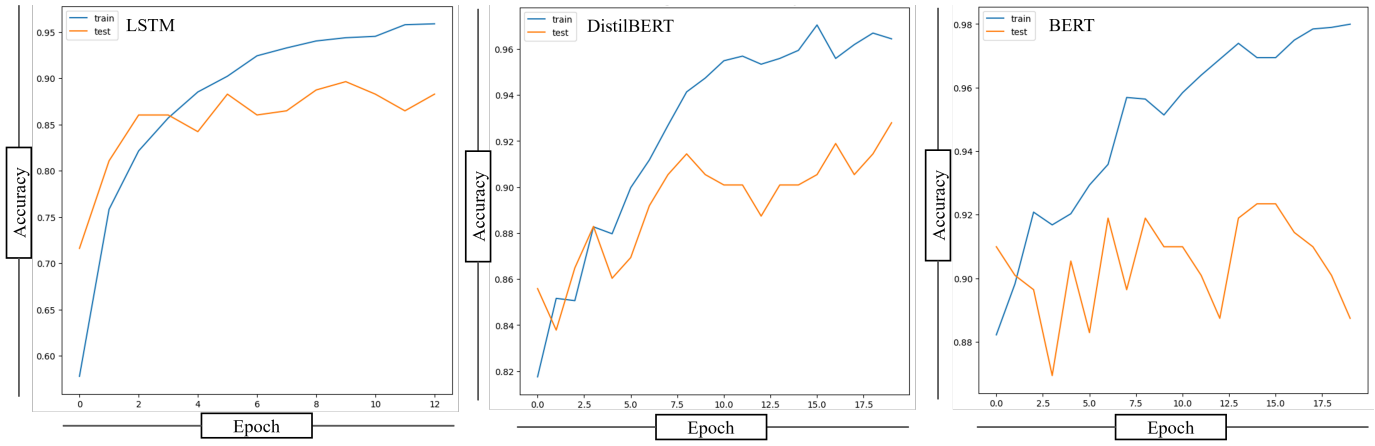



Fig. 2. Training and validation accuracy plots for LSTM, DistilBERT, and BERT models over 20 epochs. All models demonstrate increasing accuracy during training, with BERT achieving the highest peak validation accuracy. However, BERT also exhibits the highest variance in validation accuracy.

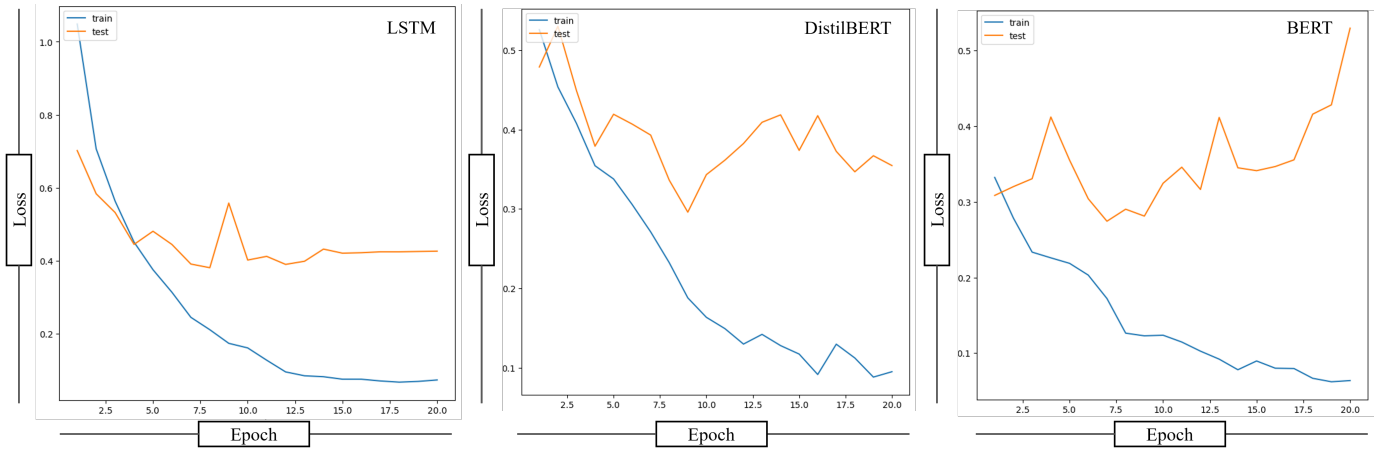



Fig. 3. Training and validation loss plots for LSTM, DistilBERT, and BERT models over 20 epochs. While all models show a decreasing trend in validation loss, BERT consistently outperforms the other models in terms of minimum validation loss. However, BERT's validation loss also fluctuates more significantly, indicating potential instability in its learning process.

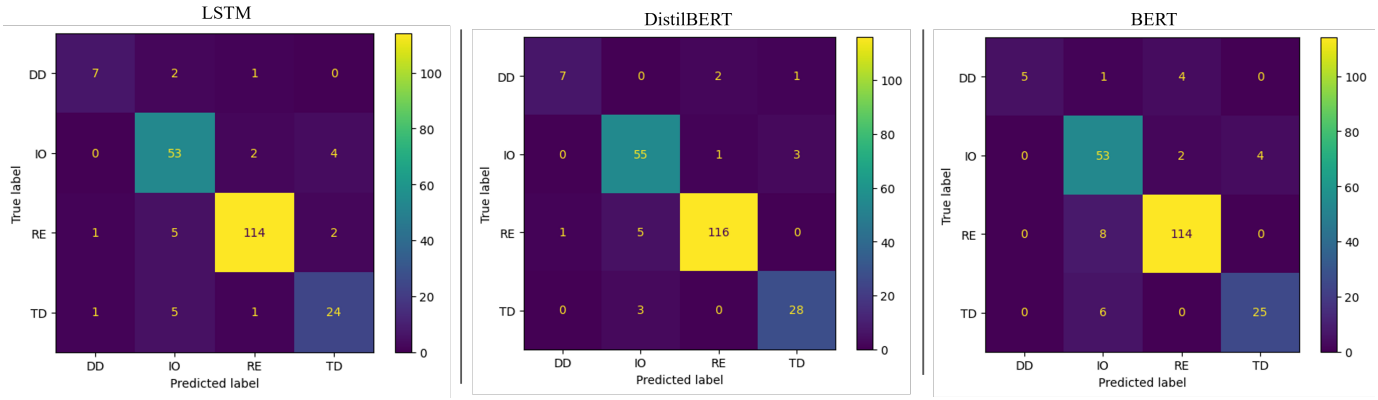



Fig. 4. Confusion matrices for LSTM, DistilBERT, and BERT models, visualizing the distribution of true and predicted labels. All models demonstrate a strong bias toward predicting the Timestamp Dependency (TD) class, with BERT showing the highest accuracy for the Reentrancy (RE) class.



Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 12:59:33 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PRECISION, RECALL, F1-SCORE, AND SUPPORT FOR EACH VULNERABILITY CLASS (DD, IO, RE, TD) AND OVERALL MACRO AND WEIGHTED AVERAGES FOR LSTM, DISTILBERT, AND BERT MODELS.

| Model | DD (Dangerous Delegatecall) Precision / Recall / F1 Score / Support | IO (Integer Overflow) Precision / Recall / F1 Score / Support | RE (Reentrancy) Precision / Recall / F1 Score / Support | TD (Timestamp Dependency) Precision / Recall / F1 Score / Support |
|---|---|---|---|---|
| LSTM | 0.78 / 0.70 / 0.74 / 10 | 0.82 / 0.90 / 0.85 / 59 | 0.97 / 0.93 / 0.95 / 122 | 0.80 / 0.77 / 0.79 / 31 |
| DistilBERT | 0.88 / 0.70 / 0.78 / 10 | 0.87 / 0.93 / 0.90 / 59 | 0.97 / 0.95 / 0.96 / 122 | 0.88 / 0.90 / 0.89 / 31 |
| BERT | 1.00 / 0.50 / 0.67 / 10 | 0.78 / 0.90 / 0.83 / 59 | 0.95 / 0.93 / 0.94 / 122 | 0.86 / 0.81 / 0.83 / 31 |
| Accuracy | 0.89 | 0.93 | 0.89 | 0.93 |
| Macro Avg | 0.84 / 0.83 / 0.83 / 222 | 0.90 / 0.87 / 0.88 / 222 | 0.90 / 0.78 / 0.82 / 222 | 0.89 / 0.89 / 0.89 / 222 |
| Weighted Avg | 0.89 / 0.89 / 0.89 / 222 | 0.93 / 0.93 / 0.93 / 222 | 0.89 / 0.89 / 0.89 / 222 | 0.89 / 0.89 / 0.89 / 222 |

performance helped adjust the dataset and model parameters, enabling continuous improvement and fine-tuning of detection.

With a proper methodology, it is natural to move forward toward the detection of smart contracts' vulnerabilities with the infusion of mixed traditions of machine-learning methods and large language models.

## V. RESULTS

The results are arranged in several critical findings stemming from the analyses performed by ML and LLMs, which exposed vulnerabilities from other smart contract syntactic-literal-based techniques. The evaluation compares the LSTM, DistilBERT, and BERT models on a labeled dataset of smart contracts, with annotated vulnerabilities shown in Table I. DistilBERT generally outperforms LSTM and BERT across almost all metrics, especially in terms of macro and weighted averages. This would imply that DistilBERT offers a more balanced performance across the different vulnerability classes. BERT achieved relatively high precision for the "DD" class but low recall and F1-score, indicating that it may be better at identifying actual true positives but constantly struggles against false positives.

The LSTM model was classified among high-poised ones, giving it an overall accuracy of 89% that ensured it was upheld by a reasonably rich classification report with sufficiently high precision (0.97 on the 'RE' class) and good recall on most classes. It indicates from the confusion matrix of Figure 4 that it performed particularly well by predicting the 'RE' class correctly (114 correct of 122) with only minor misclassifications in the other categories, such as 'IO' and 'TD.' In parallel, while the training loss noticeably decreased with every epoch, the validation loss became even more steady after the tenth epoch, assuring quite good generalization with very little overfitting.

The overall performance of the DistilBERT model was the highest, standing at 87% (test loss: 0.296 in Figure 3), thus signifying good performance. From the provided classification report, the model did indeed score high marks in predicting the 'RE' class, attaining a precision of 0.98 and an F1 score of 0.96 while the performance in 'DC' and 'TD' classes was a lot less. Training progress indicates an upward trend with steady increases in accuracy through epochs, terminating at maximum training accuracy of 93.4% although validating accuracy stabilized around 86-87%. in which learning is quite effective although with some fluctuations in validation loss.

The BERT model performed satisfactorily, achieving 89% accuracy and 0.53-loss on their test in Figure 2 and Figure 3 . The maximum validation accuracy during training was recorded at 92.3%. The classification report indicates particularly high F1-scores concerning the RE class, while the DD class performed far lesser at an F1-score of 0.67, likely due to underrepresented support. On the whole, the loss of the model showed the way down through the training period, except for the last few epochs during which the validation loss showed a steep rise, indicating a probable case of overfitting.

To elaborate further, the confusion matrix (BERT) has some class confusion, especially with class RE and class TD misclassifications, but overall, class RE is well classified. Another matrix (DeBERTa) has achieved improved accuracy across most categories, most notably with reduced errors in the DD and TD classes, whereas the LSTM is almost equal to BERT on prediction accuracies but has slightly more misclassifications in TD, indicating that it struggles in distinguishing that class.

## VI. DISCUSSION

This paper contrasts the issue of smart contract vulnerability detection, creating a comparative analysis through: Classical ML models (LSTM) and well-tuned LLMs (DistilBERT and BERT). Multiple valuable recommendations surfaced in tandem with research questions.

(a) The efficiency of Models (RQ-1): Obtaining an accuracy of 89% with LSTM shows performable results that rival the other, like BERT, thereby offering a good performance. This same model shows regular training with low overfitting and great generalizability in classifying "RE" samples properly. Vulnerabilities that are more context dependent like "TD" were better covered by LLMs. DistilBERT provided relatively better results in demanding "RE," with the maximum accuracy of 87% and F1-scores. BERT achieved 89% accuracy, but with overfitting problems losing validation at later epochs. The LLMs pattern more complex than LSTM in complicated pattern capturing and better able to accommodate delicate context-rich code dependencies.

(b) Code Patterns Capture (RQ-2): LLMs were thus shown under to be superior at detecting hidden patterns within the smart contracts, utilizing their already-trained knowl-


Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 12:59:33 UTC from IEEE Xplore. Restrictions apply.

edge to detect loopholes one thing which the LSTM may fail to do. This showed itself with better precision on the 'RE' class, having a greater handling of the dependencies such that there remained difficulty with underrepresented classes, such as 'DD', signifying the need for other sources of samples from variably distributed objects.

(c) Effect of Fine-tuning (RQ-3): Fine-tuning LMs on the labeled dataset significantly improved performances of the models, achieving a relatively higher precision-and-recall than that done by LSTMs. However, the noted overfitting on BERT suggests that a larger, diverse dataset would help lower the limit of generalizability. Future work will entail augmentation of the dataset and attempt among many other LMs to sidestep these shortcomings.

Overall, the evaluated LLMs displayed the most promise for smart contract analysis surpassing the traditional ML models in complex vulnerability detection, thus offering promising venues for future investigation.

## VII. Future Work

This work paves the way for large-scale ML and LLM-based detection of smart contract vulnerabilities, with robustness. These results demonstrate that the method of LLMs can address complexity per smart contracts as DistilBERT and BERT are very specific together thus displaying promising peaks, more than what may be achieved by conventional methods. However, the study is still in progress, and some improvements are expected as **Hybrid Approach:** Bidirectional Mixing of Labeled & Unlabeled Data approach [31] in both directions. It contains the dataset expansion which adds more smart contracts as labeled and unlabeled. This is also to minimize the impact of classes that are underrepresented so as to aid in the generalization of our models. **Exploration of New LLMs:** Evaluating new Large Language Models like GPT-4 [32], RoBERTa [33] and additional domain adapted models for their ability to detect or locate vulnerabilities in the context of more complex and contextual code patterns. Semi-Supervised Learning: Working with unlabeled data with semi-supervised learning could help the different LLM models to improve their richer representation and reduce overfitting issues. **Explainability and Interpretability:** Evaluating the model with Explainable AI [34] will provide a in detailed insight of the model's prediction and classification which will help us to determine how much rely upon the trust on these models. **Applicability:** In the application-level implementation [35], a UX design prototype and backend system architecture for real-time monitoring of vulnerability detection in smart contracts could be developed, enabling practical integration into decentralized applications. Overall, the preliminary results show that LLM can complement traditional ML models in smart contract analysis, providing a good direction for future research and a useful tool to improve blockchain security.

## VIII. Conclusion

This research explored the classical machine learning (LSTM) models and the fine-tuned large language models (DistilBERT and BERT) respectively in detecting the vulnerabilities in smart contracts. The outcomes point out that one hand, LSTM models prove to be strong in performance and generalization, and on the other hand, LLMs such as DistilBERT and BERT are better in recognizing complex code patterns and context-specific vulnerabilities. The best overall accuracy was obtained by DistilBERT, while BERT demonstrated some abilities but was slightly overfitted. The results show the benefits of utilizing LLMs for automatic smart contract analysis, moreover, in capturing the dependencies that the typical models may not be able to capture. Even so, the difficulties seen in addressing neglected groups indicate the importance of a more diverse dataset along with enhanced training strategies. The future directions will be data expansion, LLM exploration, and semi-supervised learning for boosting model robustness and generalization. Consequently, this would mean that LLMs that have been opportunely fine-tuned on labeled smart contract data will contribute greatly to vulnerability detection, which in turn opens a path for the development of more reliable and efficient security analysis tools in blockchain systems.

## References


[1] R. Kiani and V. S. Sheng, "Ethereum smart contract vulnerability detection and machine learning-driven solutions: A systematic literature review," *Electronics*, Jun. 2024. [Online]. Available: https://www.mdpi.com/2079-9292/13/12/2295

[2] O. Sürücü, U. Yeprem, C. Wilkinson, W. Hilal, S. A. Gadsden, J. Yawney, N. Alsadi, and A. Giuliano, "A survey on ethereum smart contract vulnerability detection using machine learning," in *Disruptive Technologies in Information Sciences VI*, M. Blowers, R. D. Hall, and V. R. Dasari, Eds., vol. 12117, International Society for Optics and Photonics. SPIE, 2022, p. 121170C. [Online]. Available: https://doi.org/10.1117/12.2618899

[3] A. Ghaleb and K. Pattabiraman, "How effective are smart contract analysis tools? evaluating smart contract static analysis tools using bug injection," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2020, pp. 415–427. [Online]. Available: https://doi.org/10.1145/3395363.3397385

[4] Y. Sun and L. Gu, "Attention-based machine learning model for smart contract vulnerability detection," *Journal of Physics: Conference Series*, 2021. [Online]. Available: https://dx.doi.org/10.1088/1742-6596/1820/1/012004

[5] Q. Zhou, K. Zheng, K. Zhang, L. Hou, and X. Wang, "Vulnerability analysis of smart contract for blockchain-based iot applications: a machine learning approach," *IEEE Internet of Things Journal*, vol. 9, no. 24, pp. 24 695–24 707, 2022. [Online]. Available: https://doi.org/10.1109/JIOT.2022.3196269

[6] X. Tang, Y. Du, A. Lai, Z. Zhang, and L. Shi, "Deep learning-based solution for smart contract vulnerabilities detection," *Scientific Reports*, vol. 13, no. 1, p. 20106, 2023. [Online]. Available: https://doi.org/10.1038/s41598-023-47219-0

[7] B. Boi, C. Esposito, and S. Lee, "Smart contract vulnerability detection: The role of large language model (llm)," *ACM SIGAPP Applied Computing Review*, vol. 24, no. 2, pp. 19–29, 2024. [Online]. Available: https://doi.org/10.1145/3687251.3687253

[8] J. Kim, S. Lee, H. Kim *et al.*, "Robust vulnerability detection in solidity-based ethereum smart contracts using fine-tuned transformer encoder models," *IEEE Access*, 2024. [Online]. Available: https://doi.org/10.1109/ACCESS.2024.3482389

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: url="https://doi.org/10.1007/978-3-642-24797-2_4"

[10] V. Sanh, "Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019. [Online]. Available: http://arxiv.org/abs/1910.01108




Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 12:59:33 UTC from IEEE Xplore. Restrictions apply.

[11] J. Devlin, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018. [Online]. Available: http://arxiv.org/abs/1810.04805

[12] N. F. Samreen and M. H. Alalfi, "Reentrancy vulnerability identification in ethereum smart contracts," in *2020 IEEE International Workshop on Blockchain Oriented Software Engineering (IWBOSE)*. IEEE, 2020, pp. 22–29. [Online]. Available: https://doi.org/10.1109/IWBOSE50093.2020.9050260

[13] E. Lai and W. Luo, "Static analysis of integer overflow of smart contracts in ethereum," in *Proceedings of the 2020 4th International Conference on Cryptography, Security and Privacy*, 2020, pp. 110–115. [Online]. Available: https://doi.org/10.1145/3377644.3377650

[14] M. Wohrer and U. Zdun, "Smart contracts: security patterns in the ethereum ecosystem and solidity," in *2018 International Workshop on Blockchain Oriented Software Engineering (IWBOSE)*. IEEE, 2018, pp. 2–8. [Online]. Available: https://doi.org/10.1109/IWBOSE.2018.8327565

[15] J. Feist, G. Grieco, and A. Groce, "Slither: a static analysis framework for smart contracts," in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, 2019, pp. 8–15. [Online]. Available: https://doi.org/10.1109/WETSEB.2019.00008

[16] H. Yang, J. Zhang, X. Gu, and Z. Cui, "Smart contract vulnerability detection based on abstract syntax tree," *2022 8th International Symposium on System Security, Safety, and Reliability (ISSSR)*, 2022. [Online]. Available: https://doi.org/10.1109/ISSSR56778.2022.00032

[17] R. Kiani and V. S. Sheng, "Ethereum smart contract vulnerability detection and machine learning-driven solutions: A systematic literature review," *Electronics*, vol. 13, no. 12, 2024. [Online]. Available: https://www.mdpi.com/2079-9292/13/12/2295

[18] L. Prifti, B. Çiço, and D. A. Karras, "Smart contract vulnerability detection using deep learning algorithms on evm bytecode," *2024 13th Mediterranean Conference on Embedded Computing (MECO)*, 2024. [Online]. Available: https://doi.org/10.1109/MECO62516.2024.10577852

[19] Z. Wang, Q. Zheng, and Y. Sun, "Gvd-net: Graph embedding-based machine learning model for smart contract vulnerability detection," *2022 International Conference on Algorithms, Data Mining, and Information Technology (ADMIT)*, 2022. [Online]. Available: https://doi.org/10.1109/ADMIT57209.2022.00024

[20] D. He, R. Wu, X. Li, S. Chan, and M. Guizani, "Detection of vulnerabilities of blockchain smart contracts," *IEEE Internet of Things Journal*, vol. 10, no. 14, pp. 12 178–12 185, 2023. [Online]. Available: https://doi.org/10.1109/JIOT.2023.3241544

[21] Y. Zhuang, Z. Liu, P. Qian, Q. Liu, X. Wang, and Q. He, "Smart contract vulnerability detection using graph neural networks," in *Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence*, 2021, pp. 3283–3290. [Online]. Available: https://doi.org/10.24963/ijcai.2020/454

[22] J. Su, H.-N. Dai, L. Zhao, Z. Zheng, and X. Luo, "Effectively generating vulnerable transaction sequences in smart contracts with reinforcement learning-guided fuzzing," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–12. [Online]. Available: https://doi.org/10.1145/3551349.3560429

[23] K. Gong, X. Song, N. Wang, C. Wang, and H. Zhu, "Scgformer: Smart contract vulnerability detection based on control flow graph and transformer," *IET Blockchain*, 2023. [Online]. Available: https://doi.org/10.1049/blc2.12046

[24] W. Ma, D. Wu, Y. Sun, T. Wang, S. Liu, J. Zhang, Y. Xue, and Y. Liu, "Combining fine-tuning and llm-based agents for intuitive smart contract auditing with justifications," *arXiv preprint arXiv:2403.16073*, 2024. [Online]. Available: https://arxiv.org/abs/2403.16073

[25] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang *et al.*, "Codebert: A pre-trained model for programming and natural languages," *arXiv preprint arXiv:2002.08155*, 2020. [Online]. Available: https://arxiv.org/abs/2002.08155

[26] Z. Liu, P. Qian, J. Yang, L. Liu, X. Xu, Q. He, and X. Zhang, "Rethinking smart contract fuzzing: Fuzzing with invocation ordering and important branch revisiting," *arXiv preprint arXiv:2301.03943*, 2023. [Online]. Available: https://doi.org/10.1109/TIFS.2023.3237370

[27] J. F. Ferreira, P. Cruz, T. Durieux, and R. Abreu, "Smartbugs: A framework to analyze solidity smart contracts," in *Proceedings of the 35th IEEE/ACM international conference on automated software engineering*, 2020, pp. 1349–1352. [Online]. Available: https://doi.org/10.1145/3324884.3415298

[28] A. Sherstinsky, "Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network," *Physica D: Nonlinear Phenomena*, vol. 404, p. 132306, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167278919305974

[29] G. Liu and J. Guo, "Bidirectional lstm with attention mechanism and convolutional layer for text classification," *Neurocomputing*, vol. 337, pp. 325–338, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0925231219301067

[30] K. G. group. Kerasnlp models. [Online]. Available: https://keras.io/api/keras_nlp/models/

[31] X. Sun, L. Tu, J. Zhang, J. Cai, B. Li, and Y. Wang, "Assbert: Active and semi-supervised bert for smart contract vulnerability detection," *Journal of Information Security and Applications*, vol. 73, p. 103423, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S221421262300008X

[32] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023. [Online]. Available: https://arxiv.org/abs/2303.08774

[33] Y. Liu, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, vol. 364, 2019. [Online]. Available: http://arxiv.org/abs/1907.11692

[34] R. Kumar, A. Aljuhani, D. Javeed, P. Kumar, S. Islam, and A. N. Islam, "Digital twins-enabled zero touch network: A smart contract and explainable ai integrated cybersecurity framework," *Future Generation Computer Systems*, vol. 156, pp. 191–205, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167739X24000608

[35] E. Onica and M. Georgică, "Can smart contracts become smart? an overview of transaction impact on ethereum dapp engineering," in *Proceedings of the 4th International Workshop on Distributed Infrastructure for the Common Good*, 2023, pp. 31–36. [Online]. Available: https://doi.org/10.1145/3631310.3633492




Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 12:59:33 UTC from IEEE Xplore. Restrictions apply.